

Pipeline to assemble Nanopore Sequencing

Date: April 2026

Author: Vitor Gregorio

Methods

To complete this assignment, I was provided with a dataset (1_024_O.fastq.gz) and tasked with developing a pipeline using either Snakemake or Nextflow. I chose Nextflow due to my prior experience with the framework.

The first step was to “Create an environment and install tools if necessary”. I chose to use Docker, as it simplifies dependency management and avoids potential issues related to library installation. Therefore, I installed Docker using the command “*sudo apt install docker.io*” on my linux (Ubuntu 22.04) system, I then added my user to the Docker group using “*sudo usermod -aG docker \$USER*” and applied the changes with “*newgrp Docker*”. After that, it is necessary to edit or create the “/etc/docker/daemon.json” file to configure DNS settings, allowing Docker to properly resolve and download external resources. For example, the following configuration can be used: “{ “dns”: [“8.8.8.8”, “8.8.4.4”]}”.

With Docker running correctly, the second step was to “Run QC on the raw Nanopore reads” and I selected the NanoPlot tool, a tool designed to generate quality metrics and visualizations for long read sequencing data. The analysis was executed using the command “*NanoPlot -t 16 --fastq 1_024_O.fastq.gz -o nanoplot_output*” with the provided dataset as input, using 16 threads and saving results in the nanoplot_output directory.

The next step was to “Assemble the genome”. I selected the Flye assembler, a de novo assembly tool designed for a wide range of datasets, particularly long-read sequencing data generated by PacBio and Oxford Nanopore Technologies. A de novo assembler was employed due to the absence of a reference genome, as the organism being sequenced was unknown. I executed it using the command “*flye --nano-raw 1_024_O.fastq.gz --out-dir flye_output --threads 16*” using the provided dataset as input, 16 threads and saving in the flye_output folder.

The fourth step was to “QC of the assembly”, and for this purpose, I use QUAST (QUality ASsessment Tool), a tool designed to evaluate genome assemblies. I executed the command “*quast.py assemble.fasta -o quast_output -t 16*” using the genome assembled in the Flye tool, using 16 threads and saving the results in the quast_output directory.

The next step was to “Characterize the contigs”. Initially, I attempted to use BUSCO, a tool that assesses the assembly based on evolutionarily informed expectations of gene content. It searches for single copy orthologs in the determined lineage of the genome we sequenced and calculates the fractions of complete, fragmented and missing genes. So, I tried to use the tool with the parameter

“--auto-lineage”, which automatically determines the most appropriate lineage for the input genome. However, the analysis consistently failed, likely because the organism was not adequately represented in the BUSCO database.

To address this issue, I performed a BLASTn similarity search to identify the organism. The results showed 100% query coverage, indicating that the sequence corresponds to the virus Vibrio phage 1.024.O. In hindsight, I probably should have anticipated this, given the laboratory’s focus on viral genomics.

Based on this information, I used CheckV to characterize the contigs, as it is an automated tool for assessing the quality of viral genomes. First the CheckV database was downloaded. Then, the analysis was performed using the command “checkv end_to_end assemble.fasta checkv_output -t 16 -d checkv-db-v1.5”, using the genome assembled in Flye, 16 threads, the database checkv-db-v1.5, and saving the results in checkv_output folder.

Finally, I implemented the workflow in Nextflow, defining one process for each step of the pipeline. One suboptimal practice in this implementation is the inclusion of the database download as a process. Under normal circumstances, for a known species, the database should be downloaded once and provided as an input parameter to the pipeline. However, in this case, I included it as a separate process to ensure the pipeline is fully self-contained and reproducible for the purposes of this assignment. Additionally, the database is relatively small, so Nextflow downloads it during the assembly step without significantly impacting performance.

Then, I created the nextflow.config file to enable Docker support and ensure that containers run using the current user. This was configured as follows

```
“docker {  
    enabled = true  
    runOptions = '-u $(id -u):$(id -g)'  
}”
```

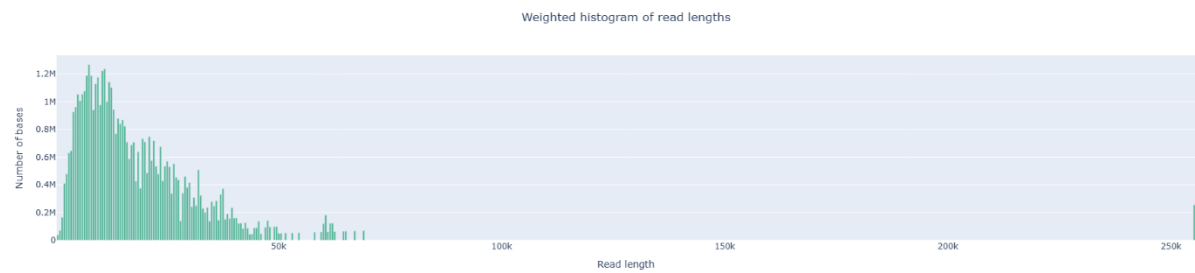
I also included MultiQC to aggregate the results from NanoPlot and QUAST, generating a final report (multiqc_report.pdf).

To execute the pipeline, the user only needs to run “nextflow run VEO_pipeline.nf --input 1_024_O.fastq.gz --threads 16 --output results”. The input dataset is specified via the --input parameter, while the number of threads can be adjusted using --threads (default: 1). The output directory can be defined with --output, which defaults to a results folder in the same directory as the script.

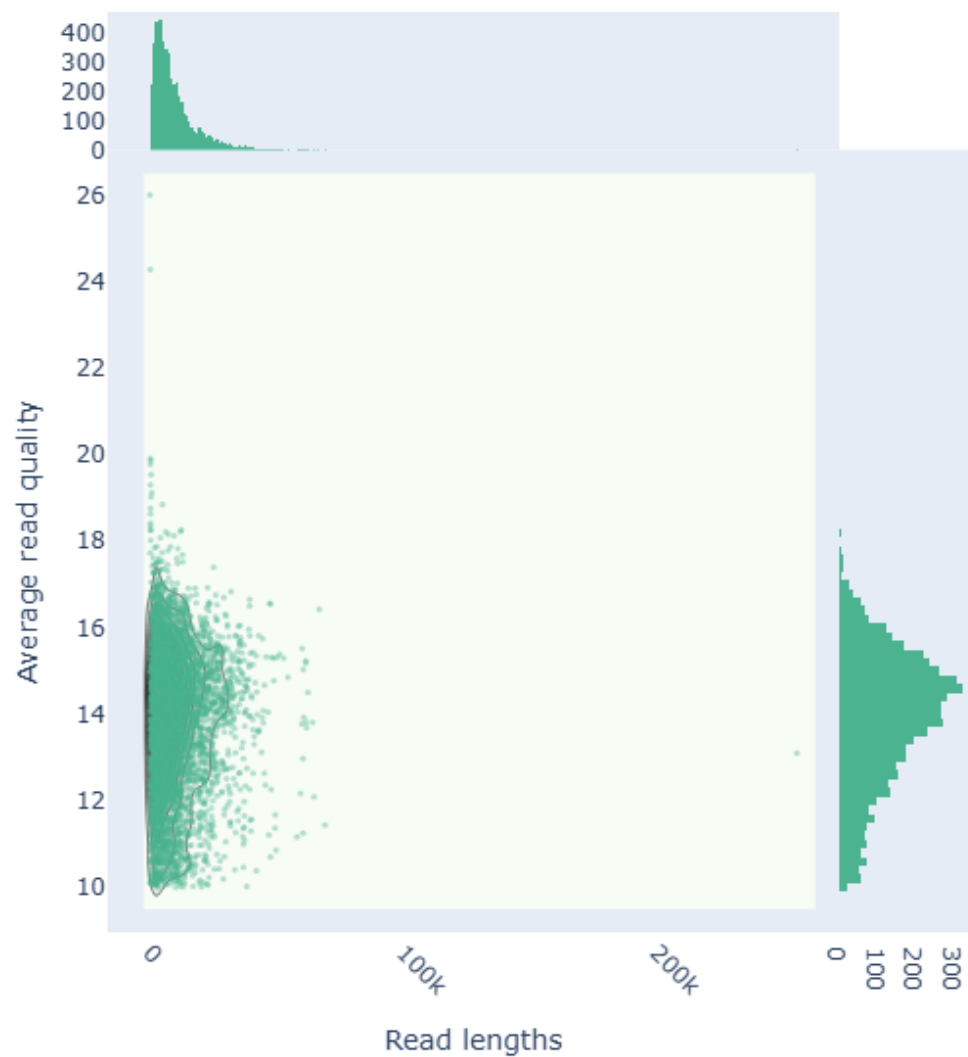
Results

The dataset provided was analyzed using NanoPlot to assess sequencing yield and quality. The sequencing run generated 50.8MB across 5251 reads. The median read quality was Q14.1, which is well within the reliable range for Oxford Nanopore technology. A significant feature of this dataset

was the read length N50 of 14,392 bp and the presence of ultra-long reads, with the longest reaching 255.2 kb. As show in the images below.



Read lengths vs Average read quality kde plot



For the assembly phase, Flye was selected as the primary assembler, running in metagenomic mode to account for potential variations in sequence abundance. The process resulted in a single circular contig of 58,829 bp with a GC content of 49.05%. An initial biological validation using BUSCO against the bacterial lineage yielded a 0% completeness score. Rather than indicating a failure in the pipeline, this discrepancy served as a diagnostic marker, suggesting that the assembled DNA was not chromosomal but rather a high-titer auxiliary genetic element.

Taxonomic identification via BLASTn confirmed the contig as a *Vibrio* Phage. To validate the integrity of this viral genome, CheckV was employed for a specialized quality assessment. The results classified the assembly as High-quality, with an estimated completeness of 96.95% and zero host contamination.